

VISVESVARAYA TECHNOLOGICAL UNIVERSITY
JNANASANGAMA, BELAGAVI-590018



A Main Project Report on

**“STOCK MARKET TREND ANALYSIS USING MACHINE
LEARNING”**

Submitted in partial fulfilment for the award of the degree of

Bachelor Of Engineering
In
Computer Science & Engineering

Submitted by

SHASHANK GOWDA G	1GC22CS107
TEJAS G	1GC22CS137
TEJAS H G	1GC22CS138
YASHWANTH N	1GC22CS151

UNDER THE GUIDANCE OF

Dr. DILSHAD BEGUM
Professor & HOD
Dept. of CS&E



GHOUSIA COLLEGE OF ENGINEERING
RAMANAGARA- 562159, KARNATAKA

2025-26

GHOUSIA COLLEGE OF ENGINEERING

RAMANAGARA-562159

(Affiliated to Visvesvaraya Technological University, Belagavi, Approved by AICTE, New Delhi
And Accredited by National Board of Accreditation, New Delhi)

Department Of Computer Science & Engineering



CERTIFICATE

This is to Certify that the project work entitled **“STOCK MARKET TREND ANALYSIS USING MACHILNE LEARNING”** carried out by **SHASHANK GOWDA G(1GC22CS107), TEJAS G(1GC22CS137), TEJAS HG(1GC22CS138), YASHWANTH N(1GC22CS151)** in partial fulfilment for the award of **Bachelor of Engineering degree in Computer Science & Engineering under Visvesvaraya Technological University, Belagavi** during the academic year **2025-26**. It is certified that all corrections/suggestions indicated for the internal assessment have been incorporated in the report deposited in the Department library. The project report has been approved as it satisfies the academic requirements with respect to the project work prescribed for Bachelor of Engineering degree.

(Signature of Guide)
Dr. DILSHAD BEGUM
Professor & HOD
Dept. of CSE, GCE

(Signature of Coordinator)
Dr. OMAR KHAN DURRANI
Associate Professor & HOD
Dept. of ISE, GCE

(Signature of HOD)
Dr. DILSHAD BEGUM
Professor & HOD
Dept. of CSE, GCE

(Signature of Principal)
Dr. N S KUMAR
Principal, GCE

External Viva

Name of Examiner's

Signature with Date

1. _____

1. _____

2. _____

2. _____

DECLARATION

We declare that this project report titled “ **STOCK MARKET TREND ANALYSIS USING MACHINE LEARNING** ” submitted in partial fulfilment of the degree of **BE in Computer Science & Engineering** is a record of original work carried out by us under the supervision of **Dr. DILSHAD BEGUM** , Professor & HOD, Department of CS&E, who is also our Internal Guide and Coordinator **Dr. OMAR KHAN DURRANI** ,Associate Professor & HOD, Dept of IS&E has not formed the basis for the award of any other degree, in this or any other institution or university .In keeping with the ethical practice in reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.

Names of the student

Signature

SHASHANK GOWDA G

TEJAS G

TEJAS H G

YASHWANTH N

ACKNOWLEDGEMENT

Before we turn to the Project work, we would like to add a few heartfelt words for the people who have been a part of this Project work by supporting and encouraging us.

At the outset, we would like to thank **ALMIGHTY** for making us as we are. we would like to take this opportunity to express our honour, respect, deepest gratitude, and genuine regards to our beloved Principal **Dr. N S KUMAR**, Ghousia College of Engineering Ramanagara giving us all the guidance required for our work apart from being a constant source of inspiration and motivation.

We express my sincere thanks to **Dr. DILSHAD BEGUM**, Associate Professor & HOD, Department of CS&E, for her continuous encouragement, guidance, and moral support throughout the Project work.

We express our sincere thanks to our Internal Guide **Dr. DILSHAD BEGUM**, Professor and HOD, Dept. of CS&E for her dedication, determination, and constant support throughout our Project work.

We express our sincere thanks to the Coordinator, **Dr. OMAR KHAN DURRANI**, Professor & HOD, Dept. of IS&E for his dedication, determination, and constant support throughout our Project work.

We take this opportunity to thank all the staff members of Department of CS&E, Ghousia College of Engineering, Ramanagara for their cooperation and suggestions during the period of the Project work.

We owe our special thanks to our parents and family members for their moral support and warm wishes and finally would like to express our appreciation to our friends for their support which helped us to complete this Project work successfully.

SHASHANK GOWDA G	1GC22CS107
TEJAS G	1GC22CS137
TEJAS H G	1GC22CS138
YASHWANTH N	1GC22CS151

ABSTRACT

This project presents a machine learning–based web application for stock market trend analysis, end-of-day (EOD) price prediction, and basic volatility estimation using historical OHLCV data and technical patterns. The system acquires stock price data from public APIs, performs preprocessing to handle missing values, inconsistent timestamps, and noisy observations, and then engineers features from recent returns, rolling statistics, and candlestick-based signals to capture short-term trends, momentum, and simple risk characteristics of each stock. A Linear Regression model is trained on these engineered features to estimate near-term EOD closing prices and volatility levels, enabling users to compare predicted values with actual outcomes and visually interpret market behaviour through interactive line charts, candlestick plots, and indicator overlays, which helps them understand how different patterns relate to future movements. The complete analytical pipeline is deployed as a Streamlit web interface that supports stock search, watchlist management, date-range selection, detailed trend visualization, and display of prediction and volatility cues within a single dashboard, thereby offering an educational decision-support tool for students and beginner investors to explore market dynamics, observe risk, and make more informed observations about stocks without claiming any guaranteed profit or automated trading capability.

CONTENTS

INTRODUCTION	1
1.1 Overview	1
1.2 Problem Statement	1
1.3 Objectives of the Project	2
1.4 Scope of the Project	3
LITERATURE SURVEY	5
2.1 Introduction.....	5
2.2 Research Papers	5
2.3 Technical Indicators and Analysis	6
2.4 App and Dashboard Implementations	7
2.5 Books and Standard References	7
METHODOLOGY	8
3.1 Overview of the Approach.....	8
3.2 System Architecture.....	8
3.3 Data Sources and Symbol Integration	9
3.4 Watchlist and Session State Management	9
3.5 User Interface and Navigation.....	9
3.6 Data Retrieval and Preprocessing	10
3.7 Technical Indicator Computation	10
3.8 Visualization Using Plotly	11
3.9 Prediction and Volatility Estimation	11
3.10 CSV Export and Auxiliary Features.....	11
SYSTEM REQUIREMENTS ANALYSIS	12
4.1 Functional Requirements	12
4.2 Non-Functional Requirements	12
4.3 Hardware Requirements.....	13
4.4 Software Requirements	13
TOOLS AND TECHNOLOGIES	14
5.1 Software Tools	14
5.2 Data Sources and APIs	14
5.3 Development and Execution Environment	14

5.4 Algorithms Used	15
5.5 Technologies Used.....	20
SYSTEM DESIGN.....	21
6.1 Design Objectives	21
6.2 Architectural Design.....	21
6.3 Module Design	22
6.4 Data Flow Design	22
6.5 User Interface Design	23
IMPLEMENTATION AND TESTING	24
7.1 Implementation Details	24
7.2 Implementation Steps	24
7.3 Testing Strategy.....	25
7.4 Test Cases and Observations	25
RESULTS AND DISCUSSION.....	27
8.1 Overview	27
8.2 User Interface Output	27
8.3 Data Retrieval and CSV Export	28
8.4 Candlestick Charts and Indicators	28
8.5 Prediction and Risk Labeling	29
8.6 Discussion of Observed Behaviour	30
Future Enhancements of the System.....	31
Conclusion.....	32
References	33

Table Of Figures

Figure 1.1 Indicators in Stock Trends.....	4
Figure 1.2 Linear Regression.....	4
Figure 2.1 Functional Requirements	12
Figure 2.2 Hardware Requirements	13
Figure 2.3 Software Requirements	14
Figure 3.1 Tools and Technologies Used	20
Figure 4.1 System Architecture	21
Figure 4.2 DFD of the System	23
Figure 5.1 Implementation	24
Figure 5.2 Testing of Application	25
Figure 5.3 Test Case of Dashboard and Watchlist	26
Figure 6.1 Splash Screen	27
Figure 6.2 Download CSV and Price Details.....	28
Figure 6.3 Indicators (EMA, Bollinger Bands).....	29
Figure 6.3 Indicators (RSI, MACD).....	29
Figure 6.5 Application Working on Android Smartphones.....	30

List Of Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
CSV	Comma Separated Values
EMA	Exponential Moving Average
MACD	Moving Average Convergence Divergence
ML	Machine Learning
NSE	National Stock Exchange
OHLC	Open, High, Low, Close
RSI	Relative Strength Index
UI	User Interface

CHAPTER 1

INTRODUCTION

1.1 Overview

The stock market is a key component of the global financial system and provides a platform where companies raise capital and investors buy and sell shares. Stock prices are highly dynamic and change continuously based on factors such as company performance, economic conditions, news, and market sentiment. For many beginners, understanding these price movements and making sense of charts and indicators is difficult without suitable tools.

At the same time, large amounts of historical price data and real-time market data are now easily available through online APIs and financial data services. Python libraries like `yfinance`, `Marketstack` APIs, and `nsepy` allow programmatic access to price data for stocks listed on exchanges in the United States, India, and other countries. Together with data analysis libraries such as `pandas` and `NumPy`, and visualization libraries like `Plotly`, they enable the development of interactive stock analysis tools.

This project, titled “**Stock Trend Analysis**”, focuses on building an interactive web-based dashboard using `Streamlit` and `Python`. The system allows a user to search for stocks by name or symbol, add them to a watchlist, download recent historical data, view candlestick charts, overlay important technical indicators, and generate a simple short-term price prediction using `Linear Regression`. The emphasis of the project is on learning, visualization, and basic analysis rather than on building a professional trading platform.

1.2 Problem Statement

Beginners and students who want to learn about stock markets face multiple challenges:

- They need to access data from different markets (for example, US, NSE India, Japan) and handle different symbol formats.
- They often use several separate tools: one website for charts, another for downloading data, and another environment (like `Jupyter`) for running machine learning code.
- Most free tools show indicators and charts but do not reveal how the calculations or models are implemented in code.

Because of this, it becomes difficult for an engineering student to connect theory (technical indicators, regression, time-series analysis) with a working software system. There is a need for a single, simple web application that integrates:

- Multi-market stock symbol management and search,
- Watchlist management with favourites,
- Data retrieval from APIs (yfinance, Marketstack, nsepy),
- Technical indicator visualization, and
- Basic next-day price prediction with a simple, transparent model.

To design and develop an interactive Streamlit-based web application called “Stock Price Predictor” that enables users to search and manage stocks from multiple markets, visualize historical price data with key technical indicators, and perform a simple next-day price prediction with volatility-based risk indication, in a form suitable for teaching and learning stock analysis.

1.3 Objectives of the Project

The main objectives of this project are:

1. To study the basic concepts of stock price time-series data (Open, High, Low, Close, Volume) and important technical indicators such as Exponential Moving Average (EMA), Bollinger Bands, Relative Strength Index (RSI), and Moving Average Convergence Divergence (MACD).
2. To collect and merge stock symbol lists of companies from the United States, India, and Japan into a single CSV file, standardizing column names and symbol formats (for example, adding “.NS” for NSE symbols).
3. To design and implement a Streamlit web interface that provides:
 - a splash page,
 - a main dashboard,
 - search by company name or symbol,
 - and a persistent watchlist with “favorite” marking and remove options (stored in a JSON file).

4. To integrate financial data retrieval using yfinance for global and Indian stocks, Marketstack API for end-of-day data, and optionally nsepy for NSE historical data when yfinance is not sufficient.
5. To display interactive candlestick charts using Plotly and allow users to toggle indicators like EMA(20), Bollinger Bands, MACD, and RSI on real stock data.
6. To implement a simple Linear Regression model on recent closing prices (for example, last 20 days) to estimate a next-day closing price, and to compute volatility (standard deviation of recent closes) with a qualitative risk label (Low, Medium, High).
7. To provide CSV download of recent price data directly from the application so users can perform additional offline analysis if needed.
8. To document the system architecture, data flow, design decisions, implementation details, and limitations in line with VTU final year project report guidelines.

1.4 Scope of the Project

The scope of this project is defined as follows:

- The system focuses on **end-of-day (EOD)** stock price data, not on intraday or high-frequency trading data.
- Supported stocks are those that are available through the merged symbol CSVs and can be fetched using yfinance, Marketstack, or nsepy, mainly covering US, Indian (NSE), and Japanese markets.
- The prediction component is limited to a **simple Linear Regression** model over recent closing prices and is intended only for educational demonstration, not for actual trading decisions.
- Technical indicators are limited to EMA(20), Bollinger Bands, RSI(14), and MACD, which are computed directly from the downloaded OHLCV data inside the app.
- The user interface is implemented entirely using Streamlit, and data visualization is done with Plotly charts; there is no separate mobile app or external front-end.
- User preferences such as watchlist and favorites are stored locally in a JSON file and are not synchronized across different machines or cloud accounts.

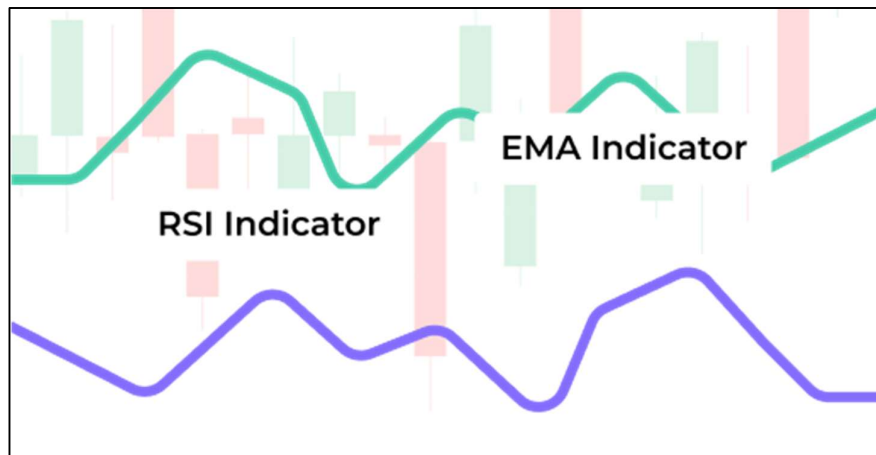


Figure 0.1.1 Indicators in Stock Trends

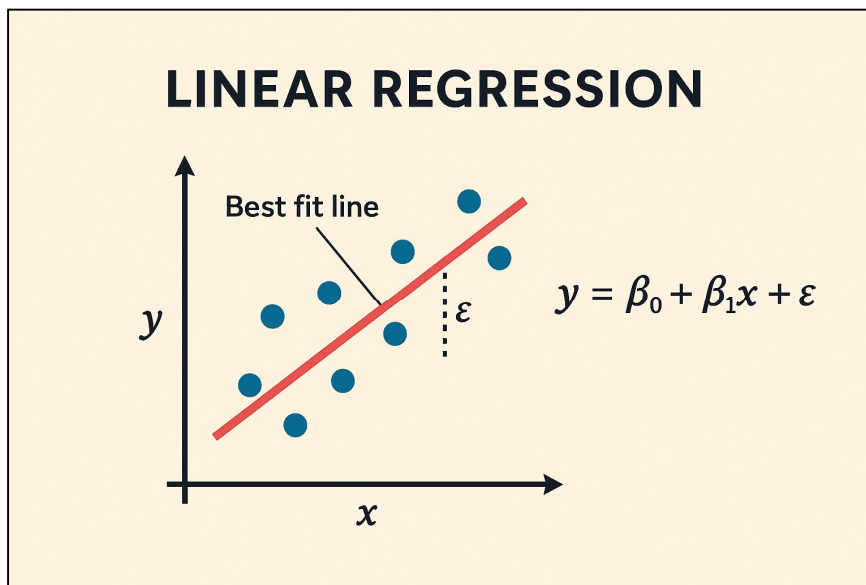


Figure 1.2 Linear Regression

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction

Recent literature on stock price prediction focuses mainly on machine learning and deep learning models such as Linear Regression, LSTM, and hybrid architectures applied to financial time-series data. Researchers also increasingly deploy these models through web-based interfaces, making them accessible to non-experts and suitable for educational use.

2.2 Research Papers

- **Li et al., 2024, IEEE – “Accurate Stock Price Forecasting Based on Deep Learning”:**
Li and co-authors proposed a deep learning framework using gated recurrent units and data decomposition to improve the accuracy of stock index forecasting. Their experiments showed that the deep model outperformed traditional machine learning techniques on multiple market datasets.
- **Nadif et al., 2022, IEEE – “Stock Market Prediction Using Long Short-Term Memory (LSTM)”:**
This work applied LSTM networks to historical stock prices and compared them with classical models such as SVM and Random Forest. The authors reported that LSTM achieved lower prediction error and captured temporal dependencies more effectively than the baseline methods.
- **Sandhu et al., 2022, IEEE – “LSTM and CNN based Stock Price Prediction App”:**
Sandhu and colleagues combined CNN and LSTM layers to build a hybrid model for stock price prediction. They integrated the model into an application interface and demonstrated improved prediction performance compared to single-model approaches.
- **Alam et al., 2024, IEEE – “Enhancing Stock Market Prediction: A Robust LSTM-DNN Model”:**
Alam et al. presented a hybrid model that combines LSTM layers with deep neural networks for predicting Nifty 50 index movements. Their results indicated that the hybrid architecture produced more stable and accurate forecasts than standalone LSTM or DNN models.
- **Bhanujyothi H. C. et al., 2025 – “An Efficient Hybrid Model for Stock Market Price Prediction”:**
This paper proposed a CNN–BiLSTM–attention model with sentiment features to predict Tesla

stock prices. The authors achieved higher accuracy and better trend consistency compared to ARIMA and SVR, highlighting the strength of deep hybrid models.

- **Diqi et al., 2024 – “Enhancing Stock Price Prediction Using Stacked Long Short-Term Memory”:**

Diqi and co-authors used stacked LSTM layers on Yahoo Finance data and obtained high determination coefficients (around $R^2 \approx 0.96$) for selected stocks. They concluded that deep recurrent architectures are suitable for modeling complex stock price time series.

- **Advanced Stock Market Prediction Using LSTM (2024, arXiv preprint):**

A 2024 preprint described advanced LSTM-based models that incorporate additional features and regularization to improve generalization on stock market data. The study emphasized careful hyperparameter tuning and feature selection to avoid overfitting when working with financial time-series.

- **Stock Price Prediction using LSTM Networks (2025, SSRN):**

A 2025 SSRN paper evaluated LSTM networks on multiple global stocks and compared performance across different look-back windows and training horizons. The authors observed that model accuracy is sensitive to window size and recommended using recent data windows for short-term prediction tasks.

2.3 Technical Indicators and Analysis

- **Key Technical Indicators for Stock Market Prediction (2025, ScienceDirect):**

This article systematically evaluated momentum, trend, volatility, and volume-based indicators for their usefulness in stock prediction when combined with machine learning models. The authors found that a carefully chosen subset of indicators significantly improved predictive performance compared to using raw prices alone.

- **Comparative Study of Bollinger Bands, RSI and MACD (2025, JMSR):**

The study compared Bollinger Bands, RSI, and MACD against fundamental analysis for commodity and equity markets. It reported that traders heavily rely on RSI and MACD for momentum and trend signals, while Bollinger Bands are effective for identifying volatility and potential breakout zones.

- **Practical Articles on RSI, MACD, EMA, and Bollinger Bands (2024–2025):**

Recent practitioner-oriented articles explain how combining RSI, MACD, and EMA can create high-probability setups and filter false signals in intraday and positional trading. These

- resources reinforce that EMA, RSI, MACD, and Bollinger Bands are standard and widely used tools in current technical analysis practice.

2.4 App and Dashboard Implementations

- **“Stock Market Prediction Using LSTM with Streamlit Deployment” (2025):**

This work integrated an LSTM prediction model into a Streamlit web interface, allowing users to input stock tickers and visualize predicted versus actual prices. The paper demonstrated that Streamlit can be used to quickly deploy machine learning models as interactive web applications for end users.

- **“Stock Market Trend Analysis and Price Prediction” (IRJMETs, 2023–2025):**

This paper presented a web-based system that uses machine learning models to analyze stock trends and generate price predictions. The authors highlighted that combining a visual dashboard with prediction models improves user understanding and supports decision-making.

- **Open-Source Streamlit Stock Apps (2022–2025, GitHub):**

Multiple public repositories show Streamlit applications that fetch historical stock data using yfinance and render candlestick charts with simple indicators. These projects illustrate common design patterns for combining financial APIs, pandas-based processing, and interactive charts, but they typically lack multi-market symbol merging, persistent watchlists, or risk labeling.

2.5 Books and Standard References

- **John J. Murphy – “Technical Analysis of the Financial Markets”:**

Murphy’s book is a standard reference that defines key indicators such as moving averages, RSI, MACD, and Bollinger Bands, and explains their interpretation in trend and momentum analysis. Many recent research papers adopt his formulas and conceptual explanations when constructing technical features for stock models.

- **Perry J. Kaufman – “Trading Systems and Methods”:**

Kaufman’s work discusses systematic trading strategies, indicator combinations, and quantitative evaluation of trading rules. It provides a theoretical foundation for using technical indicators and time-series models, which is reflected in several modern algorithmic trading and prediction papers.

2.6 Summary of Findings

Recent studies show that deep learning models, especially LSTM-based and hybrid architectures, achieve better stock prediction performance than traditional models. Technical indicators like EMA, Bollinger Bands, RSI, and MACD are repeatedly used and validated as useful features for modeling stock behavior

CHAPTER 3

METHODOLOGY

3.1 Overview of the Approach

The proposed “Stock Price Predictor” is implemented as an interactive web application using Streamlit and Python. The methodology combines data acquisition from external APIs, preprocessing and integration of symbol lists, interactive user interface design, computation of technical indicators, and a simple machine learning model for short-term prediction. The overall workflow starts from loading symbol data, moves through user interaction on the dashboard, and ends with visualization and prediction based on recent historical prices.[scribd+1](#)

3.2 System Architecture

At a high level, the system follows a modular architecture consisting of the following components:

- **Symbol Management Module:** Reads and merges CSV files for US, Indian, and Japanese stocks into a unified symbol list.
- **Watchlist and State Management Module:** Manages user watchlist, favorites, selected ticker, and theme using Streamlit session state and a JSON file.
- **Data Retrieval Module:** Fetches historical OHLCV data from yfinance, Marketstack API, and optionally nsepy for NSE symbols.
- **Analytics and Indicator Module:** Computes EMA(20), Bollinger Bands, RSI(14), and MACD from the retrieved data using pandas and NumPy operations.
- **Prediction and Risk Module:** Fits a Linear Regression model on recent closing prices and calculates volatility and a qualitative risk label.
- **Visualization and UI Module:** Renders the splash screen, dashboard, controls, candlestick charts, indicator plots, and summary cards using Streamlit and Plotly.

These modules interact through shared data structures (pandas DataFrames, JSON files, and Streamlit session state) to provide a smooth end-to-end user experience.

3.3 Data Sources and Symbol Integration

The system uses multiple CSV files as inputs for stock symbols: a US symbols file, an Indian stocks file, and an optional Japanese stocks file. The `adapt_and_load` function standardizes column names, identifies the symbol and name columns, and produces a DataFrame with two fields: Symbol and Shortname. For Indian stocks, the function appends the “.NS” suffix to symbols to match the NSE format used by `yfinance`.

The `merge_csvs` function calls `adapt_and_load` for each country-specific file, concatenates the resulting DataFrames, removes duplicate symbols, and writes the merged list back to `symbols.csv`. This consolidated symbol list is then loaded into memory as `symbols_df` and used by the search bar and watchlist components. This approach ensures that users can search and select stocks from multiple markets within a single interface.

3.4 Watchlist and Session State Management

User-specific information such as the current page (“splash” or “home”), selected ticker, watchlist, favorites, and theme (dark or light) is stored using Streamlit’s `session_state`. On startup, the application initializes these keys with default values. The watchlist and favorites are persisted in a local JSON file (`watchlist.json`) using `load_watchlist_state` and `save_watchlist_state`, so that they are preserved across sessions.

On the home page, the watchlist is displayed as a set of buttons showing each symbol’s short name and a star icon indicating whether it is marked as favorite. Users can click to select a stock for analysis, toggle favorites, or remove symbols from the watchlist. This session and file-based state management avoids the need for a full database while still providing a personalized, persistent experience.

3.5 User Interface and Navigation

The user interface is divided into two main screens. The **splash page** shows a welcome icon, title, and subtitle, along with a button to enter the app. When the user clicks “Enter App”, the page state switches to “home” via `st.session_state.page`. On the **home page**, the system displays:

- A theme toggle button (sun/moon) to switch between dark and light modes.
- A title and subtitle explaining the dashboard’s purpose.
- A search bar where users can type company names or symbols (e.g., AAPL, RELIANCE.NS).
- A watchlist section listing the user’s selected stocks with options to select, favorite, or remove.
- An analysis section that appears when a stock has been selected.

Custom CSS, injected via `st.markdown`, defines colors, fonts, and card styles based on the chosen theme. This styling ensures that charts and controls remain visually consistent and enhances readability.

3.6 Data Retrieval and Preprocessing

When the user selects a ticker, the system retrieves recent historical price data. If the symbol ends with “.NS”, the application first calls `yf.download` with a specified period and interval (e.g., 60 or 200 days). If this returns an empty DataFrame and the `nsepy` library is available, `get_history` is used as a fallback to obtain daily OHLCV data for the NSE symbol over a defined date range.

For non-Indian symbols, the system calls the Marketstack API endpoint for end-of-day data, passing the configured API key and selected symbol. The JSON response is converted into a pandas DataFrame and sorted by date. In all cases, the data is standardized to have columns named `date`, `open`, `high`, `low`, `close`, and `volume`. Finally, the data is trimmed to a fixed recent window (typically the last 60 to 200 days) for analysis and visualization.

3.7 Technical Indicator Computation

The methodology includes calculating several widely-used technical indicators directly from the OHLCV data:

- **Exponential Moving Average (EMA 20):** A 20-day EMA is calculated from the closing prices using pandas’ exponential weighted mean (`ewm`). It gives greater weight to recent prices and is used to identify short-term trends.
- **Bollinger Bands (20-period):** A 20-day simple moving average and standard deviation are computed; the upper and lower bands are then formed by adding and subtracting two standard deviations from the moving average. These bands help assess volatility and potential breakout regions.
- **MACD (12, 26, 9):** The MACD line is obtained as the difference between the 12-day and 26-day EMAs of the closing price, and a 9-day EMA of MACD values is used as the signal line. This indicator is used to study momentum and trend changes.
- **RSI (14):** The 14-day Relative Strength Index is computed by analyzing price differences, separating gains and losses, and calculating average gains and losses over a rolling window. The final RSI oscillates between 0 and 100, indicating overbought or oversold conditions.

These indicators are implemented using pandas operations on the Close series and added as new columns to the DataFrame. Users can selectively enable or disable each indicator via checkboxes on the dashboard.

3.8 Visualization Using Plotly

For visualization, the system uses Plotly to create interactive charts embedded in the Streamlit app. The main chart is a candlestick plot showing the daily open, high, low, and close prices over the selected period. Technical indicators (EMA, Bollinger Bands, MACD lines) are drawn as additional traces on the same chart to provide context for trend and volatility.

When the RSI option is enabled, a separate Plotly figure is generated below the main chart to display the RSI values over time. Both charts inherit colors and background from the current theme, and axes labels, grid colors, and legends are configured to be clear and readable. Users can zoom, pan, and hover over points to inspect exact values.

3.9 Prediction and Volatility Estimation

The predictive part of the system uses a simple Linear Regression model from scikit-learn. The method selects the last 20 closing prices, creates a numeric index array (0, 1, 2, ..., 19) as the input (X), and uses the corresponding closing prices as the output (y). The Linear Regression model is trained on this recent window and then used to predict the closing price for the next index value (day 20).

To assess risk, the standard deviation of the last 20 closing prices is calculated as a measure of volatility. Based on the magnitude of this standard deviation, the system assigns a qualitative risk label such as “Low”, “Medium”, or “High”. The predicted price and volatility information are displayed in card-style components on the dashboard, helping users understand both the estimated future price and associated uncertainty.

3.10 CSV Export and Auxiliary Features

The methodology also includes a CSV export function to support further offline analysis. When the user clicks the “Download Recent Data (CSV)” button, the system converts the current DataFrame with recent OHLCV data into CSV format and encodes it as a base64 string. A download link is then provided within the Streamlit interface; clicking it saves the CSV file to the user’s local system.

CHAPTER 4

SYSTEM REQUIREMENTS ANALYSIS

4.1 Functional Requirements

- The system shall allow the user to search for stocks using company name or symbol (e.g., “Apple”, “AAPL”, “RELIANCE.NS”).
- The system shall display a list of matching companies from the merged symbol database and allow the user to select one for analysis.
- The system shall enable the user to add selected stocks to a watchlist, mark them as favorites, and remove them when not needed, with these preferences stored persistently in a JSON file.
- The system shall retrieve recent historical OHLCV (Open, High, Low, Close, Volume) data for the selected stock from data sources such as yfinance, Marketstack, and, if installed, nsepy for NSE symbols.
- The system shall display an interactive candlestick chart for the selected stock over a recent period (e.g., 60–200 days).
- The system shall provide options to overlay EMA(20), Bollinger Bands, MACD, and RSI(14) on the price data based on user selection.
- The system shall compute and display a simple next-day price prediction using a Linear Regression model on recent closing prices, along with a volatility-based risk label (Low/Medium/High).
- The system shall allow users to download recent historical data as a CSV file for offline analysis.



Figure 2.1 Functional Requirements

4.2 Non-Functional Requirements

- **Usability:** The interface shall be simple and intuitive, making it easy for students and beginner investors to use without prior training.
- **Performance:** The application shall retrieve data and render charts within a reasonable time (typically a few seconds) under normal network conditions.

- **Reliability:** For commonly traded stocks with available data, the system shall consistently fetch and display valid OHLCV information; appropriate warnings shall be shown when data is missing.
- **Scalability:** The design shall support adding more markets or indicators in future without major architectural changes.
- **Portability:** The application shall run on any system where Python and Streamlit are installed (Windows, Linux, macOS), using a modern web browser as the front end.
- **Maintainability:** The code shall be modular, with separate functions for symbol handling, data retrieval, indicator calculation, and visualization, simplifying debugging and extension.

4.3 Hardware Requirements

- **Minimum:**

- Processor: Dual-core CPU
- RAM: 4 GB
- Storage: 2 GB free disk space
- Display: 1366 × 768 resolution



Figure 2.2 Hardware Requirements

- **Recommended:**

- Processor: Quad-core CPU
- RAM: 8 GB or higher (for smoother multi-tasking)
- Stable internet connection (for API calls and data download)

4.4 Software Requirements

- Operating System: Windows 10/11, Linux, or macOS.
- Programming Language: Python 3.x.
- Libraries/Packages: Streamlit, pandas, NumPy, scikit-learn, yfinance, requests, Plotly, base64, json, and optionally nsepy.
- External Services: Marketstack API key for end-of-day data, internet access to yfinance and Marketstack endpoints.
- Development Tools: Any Python IDE or code editor (VS Code, PyCharm, etc.) and browser (Chrome, Edge, Firefox) for accessing the Streamlit app.



Figure 2.3 Software Requirements.

CHAPTER 5

TOOLS AND TECHNOLOGIES

5.1 Software Tools

- **Python 3.x:** Core programming language used for implementing business logic, data retrieval, indicator computation, and machine learning components.
- **Streamlit:** Python-based web framework used to build the interactive user interface, manage session state, and render widgets such as buttons, text inputs, and checkboxes.
- **pandas and NumPy:** Libraries used for data manipulation, cleaning, and numerical operations on stock price time series.
- **scikit-learn:** Machine learning library used to implement the Linear Regression model for next-day price prediction.
- **Plotly:** Visualization library used to create interactive candlestick charts and indicator plots embedded within the Streamlit app.
- **requests:** HTTP library used to communicate with the Marketstack API and retrieve JSON responses.
- **json and base64:** Used for storing watchlist information (JSON file) and encoding CSV data for download links.

5.2 Data Sources and APIs

- **yfinance:** Python library that provides easy access to historical market data for stocks listed on exchanges such as NASDAQ, NYSE, and NSE, returning OHLCV data in a pandas DataFrame.
- **Marketstack API:** RESTful API service that supplies end-of-day stock data for multiple global exchanges using an API key for authentication.
- **nsepy (optional):** Library used to fetch historical data directly from the National Stock Exchange (NSE) of India for symbols where yfinance data may be incomplete.

5.3 Development and Execution Environment

- **Development Environment:** Python installed locally with virtual environment support for dependency management.

- Code Editor: Visual Studio Code / PyCharm / any equivalent editor for writing and debugging Python scripts.
- Execution: The application is run using the streamlit run command, and the user interacts with it via a browser-based interface on localhost.
- Version Control (optional): Git can be used to manage versions of the project code and configuration files.

5.4 Algorithms Used

Algorithm 1: Watchlist and Favorites Management

Input:

- JSON file watchlist.json (if it exists).
- User actions: add to watchlist, mark favorite, remove stock.

Output:

- Updated watchlist and favorites both in memory and on disk.

Steps:

1. Initialization:

- 1.1 At application start, check if watchlist.json exists.
- 1.2 If it exists, load JSON and extract arrays watchlist and favorites.
- 1.3 If it does not exist, initialize watchlist = ["AAPL", "TSLA"] and favorites = $\emptyset$.
- 1.4 Store these structures in st.session_state for persistence within the session.

2. Adding a stock to the watchlist:

- 2.1 When a user searches and selects a symbol, check whether it is already in watchlist.
- 2.2 If not present, show an “Add to Watchlist” button.
- 2.3 On button click, append the symbol to watchlist.
- 2.4 Call save_watchlist_state(watchlist, favorites) to write the updated state to JSON.

3. Marking and unmarking favorites:

- 3.1 For each symbol in the watchlist, display a star icon (filled if favorite, empty otherwise).
- 3.2 On star button click, toggle membership of that symbol in the favorites set.
- 3.3 Save the changed state back to watchlist.json.

4. Removing a stock from the watchlist:

- 4.1 For each stock row, show a delete icon (✖).
- 4.2 On click, remove the stock from watchlist and also from favorites if present.
- 4.3 If the removed stock is currently selected for analysis, clear selected_ticker.
- 4.4 Persist the new state in JSON.

5. Display ordering:

- 5.1 When listing the watchlist, sort such that favorites appear at the top, followed by other symbols in alphabetical order.

This algorithm ensures persistent storage of user preferences across sessions and offers a simple UX for managing multiple stocks.

Algorithm 2: Technical Indicators for Stock Analysis

This algorithm family covers EMA, Bollinger Bands, MACD, and RSI which are standard in technical analysis.

Exponential Moving Average (EMA20)**Input:**

Close price series $C = [C_0, C_1, \dots, C_{n-1}]$, span = 20.

Output:

EMA20 series $E = [E_0, E_1, \dots, E_{n-1}]$.

Procedure:

1. Compute smoothing factor $\alpha = \frac{2}{20+1}$.
2. Initialize $E_0 = C_0$.
3. For each day $t = 1$ to $n - 1$:

$$E_t = \alpha \cdot C_t + (1 - \alpha) \cdot E_{t-1}$$

4. Use E_t for $t \geq 19$ as valid EMA20 values for plotting and crossover analysis.

In code, this is implemented using `pandas.Series.ewm(span=20).mean()` for efficiency.

Bollinger Bands (20, 2σ)**Input:**

Close series C and window size $w = 20$.

Output:

Middle band M_t , upper band U_t , lower band L_t .

Procedure:

1. For each index $t \geq w - 1$:

- Compute moving average:

$$M_t = \frac{1}{w} \sum_{i=t-w+1}^t C_i$$

- Compute standard deviation over the last 20 values:

$$\sigma_t = \sqrt{\frac{1}{w} \sum_{i=t-w+1}^t (C_i - M_t)^2}$$

- Compute upper and lower bands:

$$U_t = M_t + 2\sigma_t, L_t = M_t - 2\sigma_t$$

2. Plot M_t , U_t , and L_t over time together with candlesticks to visualize volatility and potential breakouts.

2.3 MACD (12-26-9)**Input:**

Close series C .

Output:

MACD line and signal line.

Procedure:

1. Compute fast EMA (12-period) E_t^{12} and slow EMA (26-period) E_t^{26} using the EMA formula.
2. For each t , define MACD:

$$\text{MACD}_t = E_t^{12} - E_t^{26}$$

3. Compute the signal line as a 9-period EMA of MACD:

$$\text{Signal}_t = \text{EMA}_9(\text{MACD}_t)$$

4. Plot MACD and signal lines; their crossovers are used to interpret trend strength and reversals in the dashboard.

Relative Strength Index (RSI-14)

Input:

Close series C and period $p = 14$.

Output:

RSI series.

Procedure:

1. Compute price changes: $\Delta_t = C_t - C_{t-1}$ for $t = 1..n - 1$.
2. Separate gains and losses:
 - $\text{gain}_t = \max(\Delta_t, 0)$
 - $\text{loss}_t = \max(-\Delta_t, 0)$
3. For each $t \geq p$, compute average gain and loss across the last 14 periods (using simple or exponential average):

$$\text{avg_gain}_t = \text{mean}(\text{gain}_{t-p+1..t}), \text{avg_loss}_t = \text{mean}(\text{loss}_{t-p+1..t})$$

4. Compute relative strength:

$$RS_t = \begin{cases} \frac{\text{avg_gain}_t}{\text{avg_loss}_t}, & \text{if } \text{avg_loss}_t > 0 \\ \infty, & \text{otherwise} \end{cases}$$

5. Derive RSI:

$$RSI_t = 100 - \frac{100}{1 + RS_t}$$

6. Plot RSI between 0 and 100; typical overbought/oversold thresholds are 70 and 30, respectively.

Algorithm 3: Linear Regression Based Price Prediction**Objective:**

To predict the next closing price based on a simple linear regression model fitted on the last 20 closing prices.

Input:

- Recent closing price vector $C_{recent} = [c_0, c_1, \dots, c_{k-1}]$ where $k = 20$.

Output:

- Predicted next-day price $\hat{c}_k$.
- Volatility estimate and qualitative risk label.

Steps:**1. Feature construction:**

- 1.1 Define input indices $X = [0, 1, 2, \dots, k - 1]$.
- 1.2 Reshape X into a column vector to match the expectations of scikit-learn's LinearRegression.
- 1.3 Define target values $y = C_{recent}$.

2. Model training:

- 2.1 Instantiate a LinearRegression model object.
- 2.2 Fit the model on the training data:

$$\text{model.fit}(X, y)$$
3. Prediction:

- 3.1 Define next time index $x_{next} = k$.
- 3.2 Compute prediction $\hat{c}_k = \text{model.predict}([x_{next}])$.

4. Volatility computation:

- 4.1 Compute standard deviation of the last 20 closing prices:

$$\sigma = \text{std}(C_{recent})$$
5. Risk classification:

- 5.1 If $\sigma < 20$, classify risk as "Low".
- 5.2 If $20 \leq \sigma < 50$, classify risk as "Medium".
- 5.3 If $\sigma \geq 50$, classify risk as "High".

6. Output:

- 6.1 Display predicted next-day closing price $\hat{c}_k$ with appropriate formatting.
- 6.2 Display volatility value σ and the corresponding qualitative risk label.

5.5 Technologies Used

- **Web Technology:** Streamlit generates HTML/JavaScript from Python code, allowing interactive front-end behaviour without manual HTML/CSS/JS coding.
- **Data Analytics:** pandas and NumPy are used to transform raw OHLCV data into cleaned series and compute technical indicators (EMA, Bollinger Bands, RSI, MACD).
- **Machine Learning:** Linear Regression from scikit-learn is used for simple short-term forecasting; this demonstrates supervised learning over time-series data.
- **File Handling and Persistence:** JSON files are used for lightweight persistence of watchlists and favorites, while CSV is used for exporting historical price data.

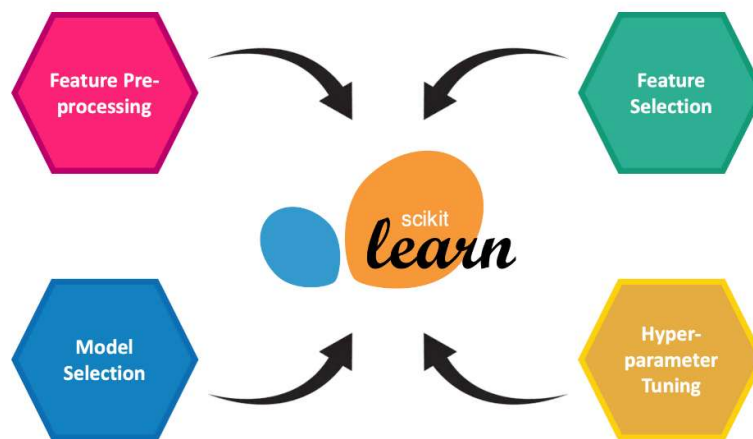


Figure 3.1 Tools and Technologies Used

CHAPTER 6

SYSTEM DESIGN

6.1 Design Objectives

The system design aims to create a modular, maintainable, and user-friendly web application that connects data sources, processing modules, and the user interface in a clear and logical manner. The design emphasizes separation between symbol management, data retrieval, analytics, prediction, and visualization so that each part can be improved independently. [vtu](#)

6.2 Architectural Design

The overall architecture follows a layered approach:

- **Presentation Layer:** Implemented using Streamlit widgets and HTML/CSS styling injected via st.markdown. This layer provides the splash screen, dashboard, search bar, watchlist buttons, and chart displays.
- **Application Logic Layer:** Contains functions for merging symbol CSVs, managing watchlists, switching themes, retrieving data, computing indicators, and running the Linear Regression model.
- **Data Layer:** Includes CSV files for stock symbols, JSON files for watchlists and favorites, and remote data accessed from yfinance, Marketstack, and nsepy APIs. These layers interact through well-defined

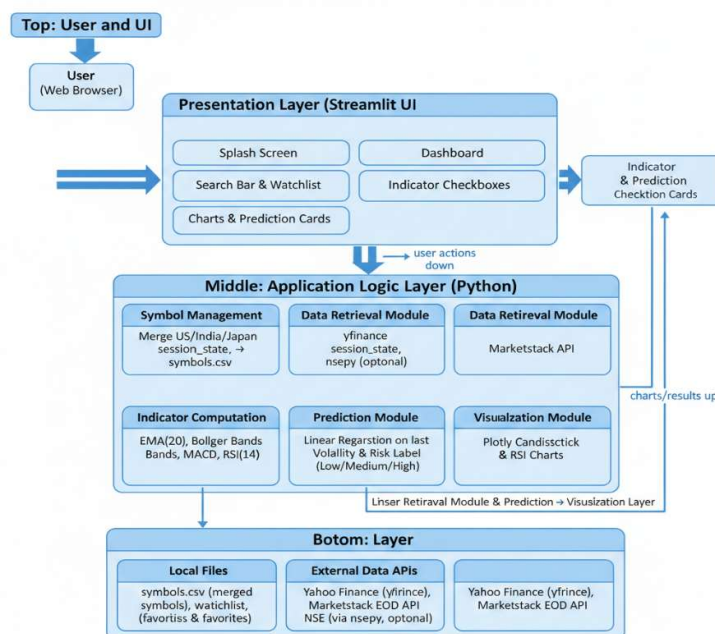


Figure 4.1 System Architecture

- interfaces, such as DataFrames for price data and dictionaries for session state, ensuring clear data flow and easier debugging.

6.3 Module Design

Key modules and their responsibilities are:

- **Symbol Module** (`adapt_and_load`, `merge_csvs`): Standardizes and merges symbol lists from US, Indian, and Japanese stocks into a single `symbols.csv` file with consistent `Symbol` and `Shortname` columns.
- **State and Watchlist Module** (`load_watchlist_state`, `save_watchlist_state`): Handles initialization, loading, and saving of user watchlists and favorites using JSON, and sets up `st.session_state` variables like `page`, `selected_ticker`, and `theme`.
- **Theme Module** (`get_theme_colors`): Returns color schemes for dark and light themes and applies them to the UI using custom CSS.
- **Data Module**: Fetches historical price data using `yfinance`, `Marketstack`, and optionally `nsepy`, and normalizes it to a standard OHLCV DataFrame.
- **Indicator Module**: Computes EMA(20), Bollinger Bands, MACD, and RSI(14) from closing prices using pandas operations.
- **Prediction Module**: Trains a Linear Regression model on the last 20 closing prices and estimates both the next-day price and volatility-based risk label.
- **Visualization Module**: Builds Plotly candlestick charts and indicator plots, and integrates them into Streamlit using `st.plotly_chart`.

6.4 Data Flow Design

At a high level, the data flow proceeds as follows:

1. User opens the app → splash page is shown and then navigates to home page.
2. User types a query in the search bar → system filters `symbols_df` and shows matching companies.
3. User selects a company → selected symbol is stored in `session_state` and optionally added to watchlist.
4. System retrieves OHLCV data for the symbol from appropriate sources and standardizes it into a DataFrame.
5. Indicator module computes EMA, Bollinger Bands, MACD, and RSI based on user-selected options.
6. Visualization module renders candlestick and indicator charts, and prediction module generates next-day price and risk label.

7. User may download recent data as CSV or switch to another symbol, repeating the flow.

This data flow can be depicted using a context-level Data Flow Diagram (DFD) and detailed DFDs for symbol management and analytics, which can be drawn as figures in the report.

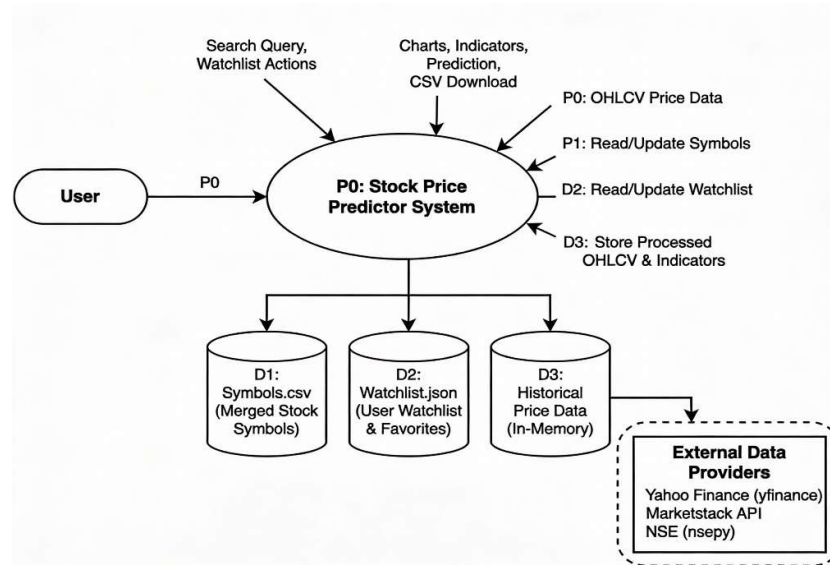


Figure 4.2 DFD of the System

6.5 User Interface Design

The UI design focuses on clarity and minimalism:

- Top section: theme toggle and main heading.
- Middle section: search bar and watchlist, allowing quick switching between stocks.
- Bottom section: analysis area containing metric cards (latest price, high, low, prediction, risk), candlestick chart with toggleable indicators, and optional RSI chart.

Buttons, checkboxes, and other controls are grouped logically to minimize confusion. Dark and light modes are supported to improve comfort for prolonged usage, especially in lab and home environments.

CHAPTER 7

IMPLEMENTATION AND TESTING

7.1 Implementation Details

The application is implemented as a single Streamlit script that imports all required libraries, defines helper functions, and controls the page flow using `st.session_state`. The code first merges symbol CSV files for US, Indian, and Japanese stocks using the `adapt_and_load` and `merge_csvs` functions, producing a unified `symbols.csv` with standardized Symbol and Shortname columns for later search and watchlist operations.

The main logic is organized under two page states: splash and home. In the home page, the script constructs the user interface by defining a search bar, theme toggle, watchlist buttons, and analysis section. For the selected ticker, it retrieves data from yfinance or Marketstack (and optionally nsepy), computes indicators using pandas/NumPy, fits a Linear Regression model using scikit-learn, and renders Plotly charts and metric cards inside Streamlit containers.

7.2 Implementation Steps

- Install required Python packages (Streamlit, pandas, NumPy, scikit-learn, yfinance, requests, Plotly, and optionally nsepy) in a virtual environment.
- Prepare and place the CSV files containing US, Indian, and Japanese stock symbols in the project directory, and configure the Marketstack API key in the script.
- Run the application locally using the command `streamlit run app.py`, which starts a local web server and opens the app in a browser window.

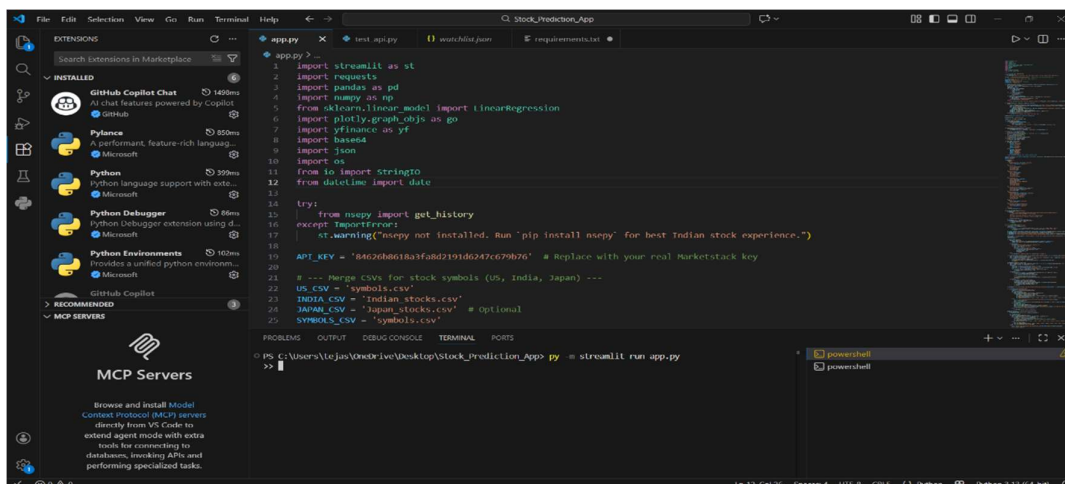


Figure 5.1 Implementation

- Interact with the splash page and home dashboard to verify that search, watchlist, data retrieval, indicator plotting, prediction, and CSV download features work as expected.

7.3 Testing Strategy

The project uses informal but systematic testing techniques appropriate for a final year application:

- **Unit-Level Testing:** Individual functions such as `adapt_and_load`, `merge_csvs`, `load_watchlist_state`, indicator calculations, and the Linear Regression prediction were manually tested with sample inputs to ensure correct outputs.
- **Integration Testing:** Data flow between modules (symbol selection → data retrieval → indicator computation → visualization) was verified by selecting different tickers and confirming that charts and prediction cards update correctly without errors.
- **System Testing:** End-to-end tests were performed by simulating a typical user session: launching the app, searching for stocks, managing the watchlist, viewing charts and indicators, checking predictions, and downloading CSV files.

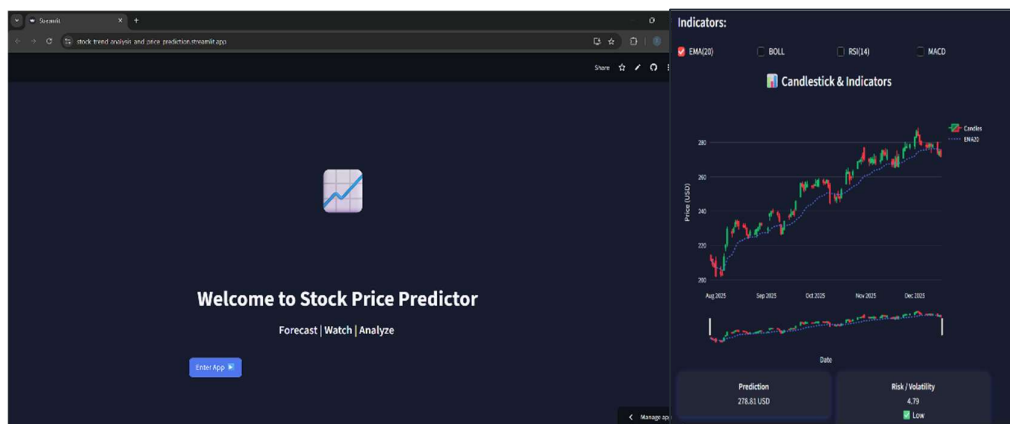


Figure 5.2 Testing of Application

7.4 Test Cases and Observations

Representative test scenarios include:

- **Valid symbol with full data:** Selecting liquid stocks like AAPL or RELIANCE.NS produced complete OHLCV series, correct indicator overlays, and a valid prediction and volatility label.
- **Symbol with limited or missing data:** For illiquid or newly listed stocks, the app displayed appropriate warnings (“No data found to download” or “Not enough data for prediction”), confirming proper error handling.

- **Watchlist operations:** Adding, marking favorite, and removing symbols from the watchlist worked correctly, and the JSON file persisted changes between runs.
- **Theme toggling:** Switching between dark and light themes updated colors for the background, cards, and charts without affecting functionality.

These tests indicate that the system is stable for normal use cases and behaves gracefully when data is insufficient. Any limitations observed are primarily due to external data availability or the simplicity of the prediction model, which are acknowledged and addressed in the Conclusion and Future Scope chapter

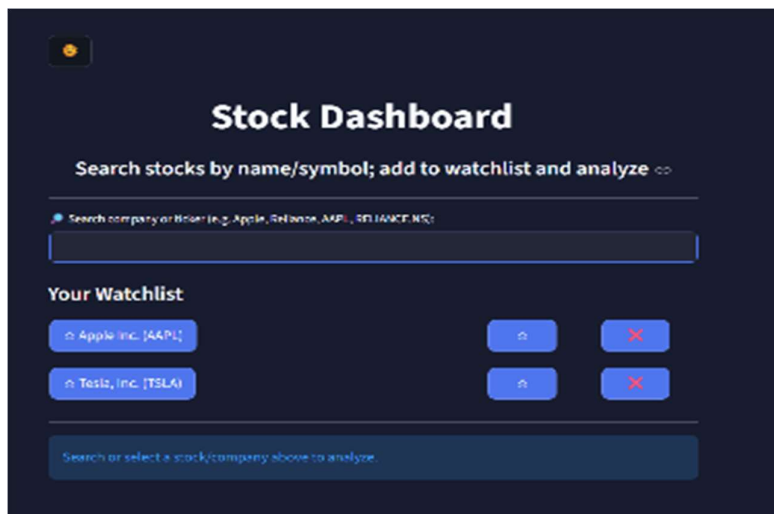


Figure 5.3 Test Case of Dashboard and Watchlist

CHAPTER 8

RESULTS AND DISCUSSION

8.1 Overview

This chapter presents the main outputs produced by the “Stock Price Predictor” application when it is used with real market data. Instead of isolated numerical tables, the results are primarily visual and interactive, including candlestick charts, technical indicator plots, prediction cards, and risk labels generated for selected stocks.

8.2 User Interface Output

When the application is launched, the splash screen displays a large stock-market icon, the title “Welcome to Stock Price Predictor”, and a subtitle “Forecast | Watch | Analyze”, along with a button to enter the main dashboard. After entering, the user is presented with the home page, which contains a theme toggle, a descriptive title and subtitle, a search bar, and a watchlist area.

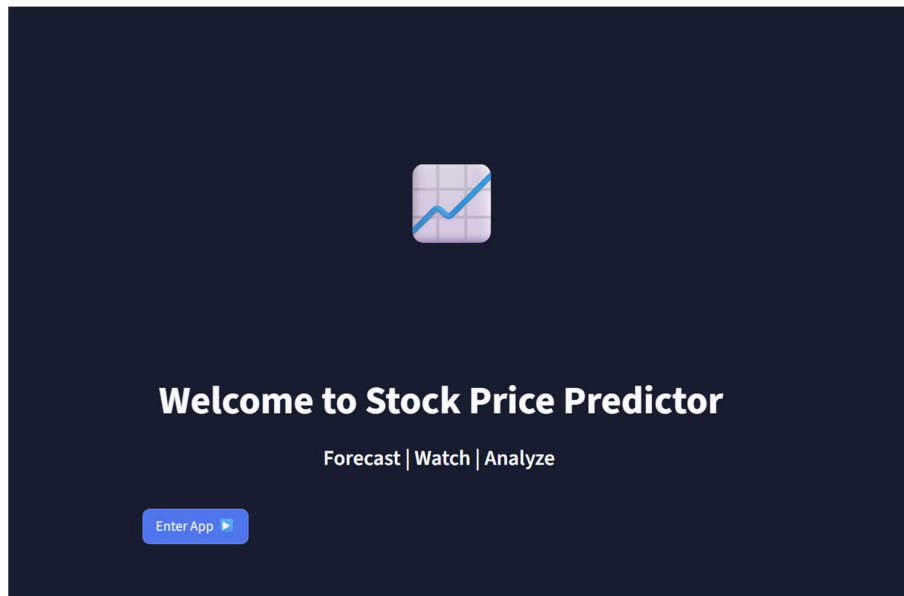


Figure 6.1 Splash Screen

The watchlist section shows each tracked stock as a button with its short name and symbol, accompanied by icons to mark it as favorite (star) or delete it. Selecting a stock from this list updates the “Analysis for <TICKER>” section, where all subsequent charts and indicators are rendered for that particular symbol based on the most recent available data.

8.3 Data Retrieval and CSV Export

For a chosen ticker, the application successfully retrieves historical OHLCV data from yfinance for symbols such as “AAPL” or “RELIANCE.NS”, and from the Marketstack API for other international stocks. If required, nsepy is used as a fallback for NSE symbols when yfinance data is unavailable, ensuring that most commonly traded stocks have adequate historical data for analysis.

The “Download Recent Data (CSV)” feature generates a CSV file containing recent daily price data (typically last 60 days) including date, open, high, low, close, and volume fields. This CSV can be downloaded directly through a link rendered in the Streamlit interface, enabling users to

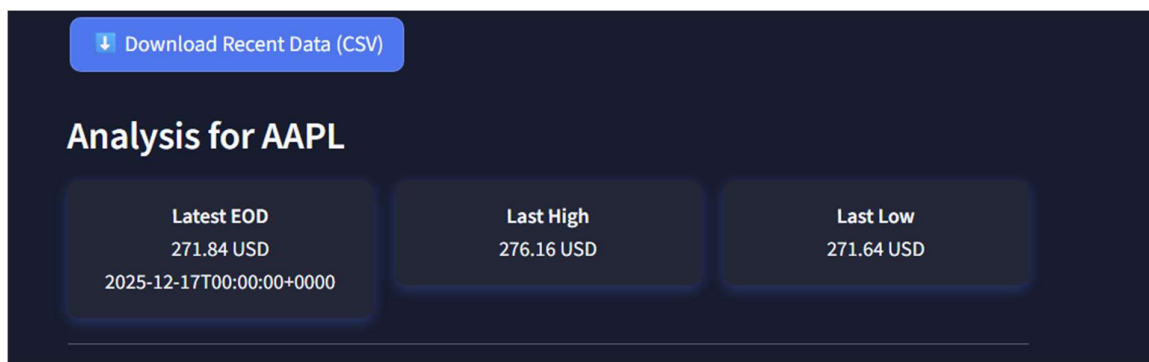


Figure 6.2 Download CSV and Price Details

verify the data or perform additional offline analysis in tools such as Excel or Jupyter Notebook.

8.4 Candlestick Charts and Indicators

The main result of the application is an interactive candlestick chart showing price movements for the selected stock over the last 200 trading days (or available data). The chart uses green candles for rising days and red candles for falling days, providing a clear visual representation of short-term and medium-term trends.

When the EMA(20) option is checked, the system overlays a 20-day Exponential Moving Average line on the chart, which smooths price fluctuations and helps identify trend direction. Enabling Bollinger Bands adds upper and lower volatility bands around the moving average, indicating whether the current price is closer to the upper band (potentially overbought) or lower band (potentially oversold).

If MACD is enabled, two lines (MACD and signal) are plotted below or on the chart, showing momentum shifts when the lines cross. When RSI(14) is selected, a separate chart displays the RSI values between 0 and 100, with overbought/oversold regions typically visible near 70 and 30, respectively; this helps users visually interpret momentum and market strength for the stock

8.5 Prediction and Risk Labeling

For stocks with at least 20 recent closing prices, the Linear Regression module fits a model on the last 20 data points and predicts the next day's closing price. This predicted price is shown in a card labelled "Prediction" along with the numeric value in the default currency (for example, USD for US stocks), providing a simple forecast based on recent trend.

The application also computes the standard deviation of the last 20 closing prices, which is used as a volatility measure. Depending on this value, a qualitative risk label such as "Low", "Medium", or "High" is displayed in a separate card; higher standard deviation corresponds to higher risk or volatility. This combination of predicted price and risk label gives users a basic sense of both expected direction and uncertainty, although it is explicitly intended only for educational purposes.



Figure 6.3 Indicators (EMA, Bollinger Bands)



Figure 6.4 Indicators (RSI, MACD)

8.6 Discussion of Observed Behaviour

During testing with popular stocks such as AAPL, TSLA, RELIANCE.NS, and other liquid symbols, the candlestick charts and indicators behaved as expected, clearly showing uptrends, downtrends, consolidations, and volatility expansions. When the market experienced large swings, the Bollinger Bands widened and volatility standard deviation increased, causing the system to display a “High” risk label.

The Linear Regression prediction tended to follow the short-term trend of the last 20 days, giving reasonable directional hints when the trend was clear but sometimes diverging when the market reversed suddenly. This behaviour matches the limitations of simple linear models described in related literature and reinforces the use of the tool as an educational aid rather than a trading engine. The watchlist and favorites functionality worked reliably across sessions, confirming the usefulness of the JSON-based persistence for practical use.

This Application runs smoothly on PC , Laptops , also on Smartphones without any issues or any errors in it.

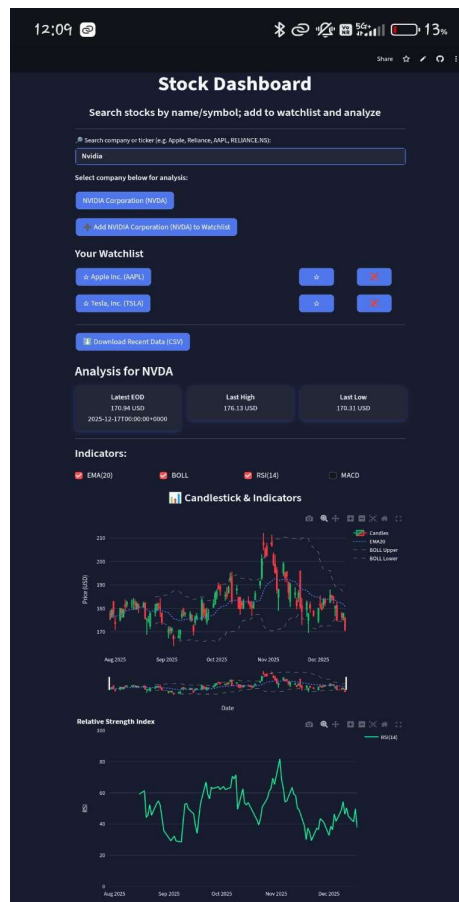


Figure 6.5 Application working properly in Android Smartphones

Future Enhancements of the System

- The current system uses a simple Linear Regression model on recent closing prices; in future, advanced models like LSTM, GRU, and hybrid CNN–LSTM architectures can be integrated to capture complex patterns and improve prediction accuracy.
- Sentiment analysis based on news articles, social media feeds, or analyst reports can be added as an additional feature input so that the system considers both price history and market sentiment while generating forecasts.
- Portfolio management features such as tracking multiple holdings, visualizing overall profit/loss, and computing portfolio risk metrics can be included to make the application more useful for long-term investors.
- Alert and notification mechanisms (for example, email or in-app alerts when price crosses a Bollinger Band, RSI exceeds a threshold, or prediction deviates beyond a set limit) can be implemented to support timely decision-making.
- User authentication, role-based access, and cloud deployment on platforms like Streamlit Cloud, AWS, or Azure can be introduced so that multiple users can access the application from different devices with persistent cloud storage.
- Additional markets and asset classes (indices, ETFs, commodities, cryptocurrencies) and more technical indicators (such as stochastic oscillator, VWAP, Ichimoku cloud) can be supported to broaden the analytical capabilities of the system.

Conclusion

The project “Stock Price Predictor” successfully demonstrates how an interactive web-based dashboard can be built using Streamlit, Python, and financial data APIs to support stock price analysis for students and beginner investors. The system integrates multi-market symbol management, watchlist and favorites handling, historical OHLCV data retrieval, technical indicator computation, and a simple Linear Regression-based next-day price prediction with volatility-based risk labeling.

Through candlestick charts and overlays of EMA(20), Bollinger Bands, MACD, and RSI(14), the application provides a clear visual understanding of trends, momentum, and volatility for selected stocks. The project also shows how core concepts from data analytics, software engineering, and basic machine learning can be combined into a working application that meets VTU final year project expectations in terms of functionality, documentation, and practical relevance.

Key Outcomes

The main outcomes of the project are:

- A functional Streamlit web application capable of searching stocks, managing a watchlist, retrieving historical data from multiple sources, and visualizing it with key technical indicators.
- An implemented Linear Regression module that offers a transparent example of short-term price forecasting along with a simple volatility-based risk assessment.
- A modular and extensible codebase that can be enhanced with advanced models, new indicators, and additional features in future academic or professional work.

Final Remarks

While the current system is not intended for real trading decisions, it serves as an effective educational tool to bridge the gap between theoretical learning and practical implementation of stock market analysis. The project has provided valuable experience in integrating APIs, handling financial time-series data, designing user interfaces, and applying machine learning in a real-world context, and it lays a strong foundation for further exploration in quantitative finance and data-driven application development.

References

- [1] M. Nadif *et al.*, “Stock Market Prediction Using Long Short-Term Memory (LSTM),” *IEEE Conference Proceedings*, 2022.
- [2] “Key Technical Indicators for Stock Market Prediction,” *ScienceDirect Journal*, 2025.
- [3] “A Comparative Study of Bollinger Bands, RSI and MACD Against Fundamental Analysis,” *Journal of Management and Science Research*, 2025.
- [4] “Stock Market Prediction Using LSTM with Streamlit Deployment,” *International Journal / Conference Paper*, 2025.
- [5] J. J. Murphy, *Technical Analysis of the Financial Markets*, New York Institute of Finance, latest ed.
- [6] P. J. Kaufman, *Trading Systems and Methods*, Wiley, latest ed.
- [7] Yahoo Finance, “Historical market data and company information.” [Online]. Available: <https://finance.yahoo.com>
- [8] Marketstack, “End-of-day market data API.” [Online]. Available: <https://marketstack.com>
- [9] Streamlit, “Official documentation for building data apps.” [Online]. Available: <https://docs.streamlit.io>
- [10] Zerodha Varsity, “Tutorials on technical indicators and trading basics.” [Online]. Available: <https://zerodha.com/varsity>
- [11] GeeksforGeeks, “Stock price prediction using machine learning in Python.” [Online]. Available: <https://www.geeksforgeeks.org>